Université Mohammed V de Rabat
École Nationale Supérieure
d'Informatique et d'Analyse des Systèmes

جامعة محمد الخامس بالرباط
Université Mohammed V de Rabat

Deep Learning – Lecture 2

Perceptron et perceptron multicouche

Pr. Aissam BERRAHOU

ENSIAS – Université Mohammed V de Rabat

Module Deep Learning – 2026-2027

Objectifs de la lecture

- décrire le neurone formel : agrégation Z puis activation $f(Z)$;
- écrire les formulations scalaire et matricielle des régressions et de la régression logistique ;
- choisir la fonction de coût (MSE, entropie croisée) ;
- expliquer la limite du perceptron simple (XOR) ;
- décrire un MLP, dénombrer ses paramètres et choisir ses activations ;
- implémenter ces modèles avec Scikit-learn et Keras.

Plan de la lecture

1. Le neurone formel

2. Régressions

3. Régression logistique

4. Le perceptron multicouche

Plan

1. Le neurone formel

2. Régressions

3. Régression logistique

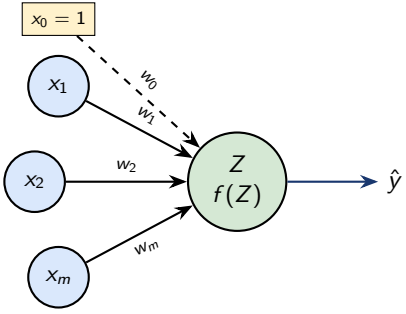
4. Le perceptron multicouche

Le perceptron

Deux étapes

1. Agrégation : $Z = \sum_{j=0}^{m} w_j x_j$, avec $x_0 = 1$ (biais w_0)
2. Activation : $\hat{y} = f(Z)$

- w_j : importance de la caractéristique j
- f : nature de la sortie (réelle, probabilité)



Plan

1. Le neurone formel

2. Régressions

3. Régression logistique

4. Le perceptron multicouche

Régression linéaire simple : prix vs surface

Surface (m²)	Prix (k€)
40	135
50	150
60	190
70	245
80	250
90	305
100	320
110	360

Activation linéaire $f(Z) = Z$

Scalaire : $\hat{y}_i = w_1 x_i + w_0$

Matricielle : $\hat{\mathbf{y}} = X\mathbf{w}$, $X = \begin{pmatrix} 40 & 1 \\ \vdots & \vdots \\ 110 & 1 \end{pmatrix}$, $\mathbf{w} = \begin{pmatrix} w_1 \\ w_0 \end{pmatrix}$

Coût : erreur quadratique moyenne

$J(\mathbf{w}) = \frac{1}{n} \sum_i (y_i - \hat{y}_i)^2$

Régression polynomiale et multiple

Polynomiale (degré m)

Entrées transformées $x, x^2, \ldots, x^m$; modèle **linéaire en w** :

$$X_{\text{poly}} = \begin{pmatrix} x_1 & x_1^2 & \cdots & x_1^m & 1 \\ \vdots & & & & \vdots \end{pmatrix}$$

Attention

Degré trop élevé $\Rightarrow$ surapprentissage.

Multiple (p variables)

$Z_i = \sum_{j=0}^{p} w_j x_{ij}$, $X \in \mathbb{R}^{n \times (p+1)}$.
Ex. : surface, chambres, âge.

Interprétation

$w_{\text{surf}} > 0$, $w_{\text{chambres}} > 0$, $w_{\text{âge}} < 0$ sur données standardisées.

Implémentations

```
from sklearn.linear_model import SGDRegressor
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import make_pipeline
pipe = make_pipeline(StandardScaler(), SGDRegressor(eta0=0.01, max_iter=1000)).fit(X, y)

import tensorflow as tf
norm = tf.keras.layers.Normalization(); norm.adapt(X)
model = tf.keras.Sequential([tf.keras.layers.Input(shape=(1,)), norm,
                             tf.keras.layers.Dense(1, activation='linear')])
model.compile(optimizer=tf.keras.optimizers.SGD(0.01), loss='mse')
model.fit(X, y, epochs=1000, verbose=0)
```

Normaliser les entrées

Sans mise à l'échelle, la descente de gradient diverge avec des surfaces ≈ 100.

Plan

1. Le neurone formel

2. Régressions

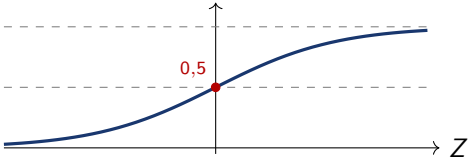
3. Régression logistique

4. Le perceptron multicouche

Sigmoïde et classification binaire

$$\hat{y} = \sigma(Z) = \frac{1}{1 + e^{-Z}} = P(y = 1 \mid \mathbf{x})$$

- seuil : $\hat{y} \geq 0{,}5 \Leftrightarrow Z \geq 0$
- $\sigma(-Z) = 1 - \sigma(Z)$
- $\sigma'(Z) = \sigma(Z)(1 - \sigma(Z))$, max 0,25



Coût : entropie croisée binaire

$J = -\frac{1}{n} \sum_i [y_i \log \hat{y}_i + (1 - y_i) \log(1 - \hat{y}_i)]$

Gradient et descente de gradient

Règle de la chaîne : $\frac{\partial \hat{y}_i}{\partial w_j} = \hat{y}_i(1-\hat{y}_i)x_{ij}$, d'où

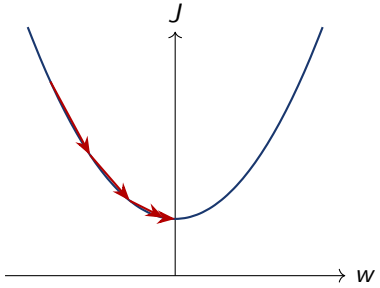
$$\frac{\partial J}{\partial w_j} = \frac{1}{n}\sum_i (\hat{y}_i - y_i)x_{ij} \qquad \Longrightarrow \qquad \nabla J = \frac{1}{n}X^T(\hat{\mathbf{y}} - \mathbf{y})$$

Mise à jour

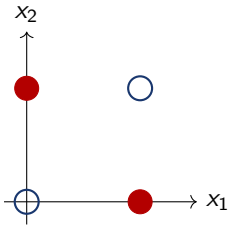
$\mathbf{w} \leftarrow \mathbf{w} - \alpha \nabla J(\mathbf{w})$

Exemple ($\alpha = 0{,}1$)

$\frac{\partial J}{\partial w} = \frac{1}{4}[0{,}38 + 1{,}1 - 0{,}93 - 0{,}72] = -0{,}0425$
$\Rightarrow w \leftarrow w + 0{,}00425$



Limite : le problème XOR



Non linéairement séparable

Aucune droite $Z = 0$ ne sépare les classes : le perceptron simple échoue.

Solution : une couche cachée

$h_1 = \mathbb{1}[x_1 + x_2 > 0{,}5]$ (OU), $h_2 = \mathbb{1}[x_1 + x_2 > 1{,}5]$ (ET)
$\hat{y} = \mathbb{1}[h_1 - h_2 > 0{,}5] = \text{XOR}$

Plan

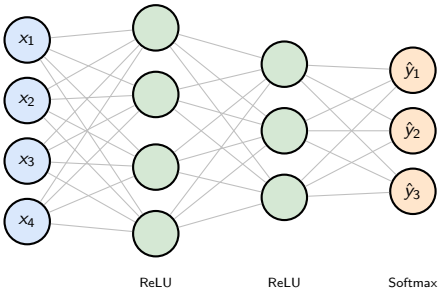
1. Le neurone formel

2. Régressions

3. Régression logistique

4. Le perceptron multicouche

Architecture d'un MLP



Couche l

$\mathbf{z}^{(l)} = W^{(l)}\mathbf{a}^{(l-1)} + \mathbf{b}^{(l)}$
$\mathbf{a}^{(l)} = g^{(l)}(\mathbf{z}^{(l)}), \quad \mathbf{a}^{(0)} = \mathbf{x}$

Paramètres

Couche dense : $n_l n_{l-1} + n_l$.
4–4–3–3 : $20 + 15 + 12 = 47$.
MNIST 784–128 : 100 480.

Approximation universelle

Une couche cachée non linéaire suffisamment large approche toute fonction continue ; la profondeur le fait avec bien moins de neurones.

Fonctions d'activation

Nom	Expression	Dérivée	Usage
Sigmoïde	$\frac{1}{1+e^{-z}}$	$\sigma(1-\sigma)$	sortie binaire, portes
Tanh	$\tanh z$	$1-\tanh^2 z$	RNN
ReLU	$\max(0,z)$	$\mathbb{1}[z>0]$	couches cachées (défaut)
Leaky ReLU	$\max(\alpha z, z)$	1 ou α	évite les neurones morts
Softmax	$e^{z_k}/\sum_j e^{z_j}$	–	sortie multiclasse

Softmax de $(2;1;0{,}1)$

$(7{,}39;\ 2{,}72;\ 1{,}11)/11{,}21 = (0{,}66;\ 0{,}24;\ 0{,}10)$

Pourquoi non linéaire ?

Sans activation : $W^{(2)}W^{(1)}\mathbf{x}$ = un seul perceptron.

MLP avec Keras

```
model = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(784,)),
    tf.keras.layers.Dense(128, activation='relu'),
    tf.keras.layers.Dense(64, activation='relu'),
    tf.keras.layers.Dense(10, activation='softmax')])
model.compile(optimizer='adam', loss='sparse_categorical_crossentropy', metrics=['accuracy'])
model.summary() # 109 386 paramètres
```

À retenir

Modèle	Entrées	Activation	Coût
Régression simple / multiple	brutes	linéaire	MSE
Polynomiale	$x, x^2, \ldots$	linéaire	MSE
Logistique	brutes	sigmoïde	entropie croisée
MLP multiclasse	brutes	ReLU + Softmax	entropie croisée

Questions de révision

1. Classer $(70, 3)$ avec $\mathbf{w} = (-6;\ 0{,}05;\ 0{,}8)$.
2. Combien de paramètres pour un MLP 784–256–128–10 ? (235 146)
3. Démontrer $\sigma' = \sigma(1 - \sigma)$.